

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 2

Industry Problem Solving Task

COURSE NAME: Deep Learning

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO 1: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. (BL-6)

Assessment Tool Weightage: 40%

QUESTIONS:

Total Marks:25

Q1. Transfer Learning

A healthcare startup aims to develop an AI-based system for detecting skin cancer from dermoscopic images. Since only a limited number of labeled images are available, design a transfer learning strategy using a pre-trained CNN model. Justify your choice of model, identify which layers should be frozen or fine-tuned, and explain how your design improves classification performance.

Q2. DenseNet and PixelNet

A smart city project requires an automated road-scene analysis system that accurately identifies roads, pedestrians, vehicles, and traffic signs at the pixel level. Design a deep learning architecture using DenseNet and PixelNet to solve this problem. Explain how the proposed architecture improves feature reuse, segmentation accuracy, and computational efficiency.

Q3. Bidirectional RNN and Encoder–Decoder

A multinational customer service company wants to build a multilingual chatbot capable of translating customer queries while preserving contextual meaning. Design an Encoder–Decoder sequence-to-sequence model using Bidirectional RNNs. Illustrate the architecture and justify how bidirectional processing improves translation accuracy.

Q4. BPTT and LSTM

A financial analytics company is developing a model to forecast stock prices using historical market data. Create an LSTM-based prediction system and explain how Backpropagation Through Time (BPTT) is used to train the network. Justify why LSTM is more suitable than a traditional RNN for this application.

Q5. Integrated Deep Learning Solution

A media streaming company plans to automatically generate captions for uploaded videos to improve accessibility. Design an integrated deep learning solution that combines Transfer Learning for visual feature extraction with an LSTM-based Encoder–Decoder architecture for

caption generation. Explain the role of each component and justify how the complete system produces accurate captions.

Code of Conduct:

I Haresh Kumar N L (192425009) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

Answers:

Q1. Transfer Learning – Skin Cancer Detection

Problem

A healthcare startup has only a limited number of labeled dermoscopic images. Therefore, training a CNN from scratch may lead to overfitting and poor performance.

Proposed Solution

Use a **pre-trained ResNet50** model trained on ImageNet and apply transfer learning.

Architecture

Dermoscopic Image → Preprocessing → ResNet50 → Global Average Pooling → Fully Connected Layer → Dropout → Softmax → Benign/Malignant

Layer Strategy

1. **Input:** Resize dermoscopic images to 224×224 pixels.
2. **Pre-trained ResNet50:** Use the model as a feature extractor.
3. **Freeze:** Initially freeze most convolutional layers because they already contain useful features such as edges, textures, and shapes.
4. **Fine-tune:** Unfreeze the last few convolutional blocks and train them using a low learning rate.
5. **Classification Layer:** Replace the original ImageNet classification layer with a new layer having two outputs:
 - Benign
 - Malignant

Why ResNet50?

ResNet50 uses **residual connections**, allowing deeper networks to learn effectively without suffering severely from vanishing gradients. Its pre-trained features are useful even when the medical dataset is small.

Performance Improvement

Transfer learning:

- Reduces training time.
- Requires less labeled data.
- Reduces overfitting.
- Reuses learned visual features.
- Improves classification accuracy compared with training from scratch

Output:

```
*** Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
94765736/94765736 0s 0us/step
Epoch 1/2
7/7 27s 2s/step - accuracy: 0.4800 - loss: 0.8771 - val_accuracy: 0.5714 - val_loss: 0.7185
Epoch 2/2
7/7 7s 1s/step - accuracy: 0.4400 - loss: 1.0032 - val_accuracy: 0.4286 - val_loss: 0.6989
Test Accuracy: 50.0 %
Prediction: Benign
```

Q2. DenseNet and PixelNet – Road Scene Analysis

Problem

The smart-city system must identify **roads, pedestrians, vehicles, and traffic signs at pixel level**, which is a semantic segmentation problem. The question specifically requires a combination of DenseNet and PixelNet.

Proposed Architecture

Road Image

↓

Preprocessing

↓

DenseNet Encoder

↓

Feature Extraction & Feature Reuse

↓

PixelNet Segmentation Decoder

↓

Pixel-wise Classification

↓

Road / Pedestrian / Vehicle / Traffic Sign

Role of DenseNet

DenseNet connects each layer with subsequent layers. This provides:

- Better feature reuse.
- Strong gradient flow.
- Reduced redundant feature learning.
- Efficient feature extraction.

Instead of learning completely new features at every layer, DenseNet reuses features learned by earlier layers.

Role of PixelNet

PixelNet can be used for **pixel-level prediction**. The extracted DenseNet features are passed to the segmentation component, which predicts a class for every pixel.

For example:

Pixel	Predicted Class
P1	Road
P2	Road
P3	Vehicle
P4	Pedestrian
P5	Traffic Sign

Advantages

Feature Reuse: DenseNet reuses information from previous layers.

Segmentation Accuracy: Pixel-level prediction provides detailed boundaries for objects.

Computational Efficiency: Feature reuse reduces unnecessary duplicate feature extraction.

Output:

```

... Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/densenet/densenet121_weights_tf_dim_ordering_tf_kernels_notop.h5
29084464/29084464 0s 0us/step
Epoch 1/2
10/10 23s 279ms/step - accuracy: 0.2510 - loss: 1.3947
Epoch 2/2
10/10 2s 201ms/step - accuracy: 0.2501 - loss: 1.3864
Segmentation Shape: (1, 128, 128)
0=Road 1=Pedestrian 2=Vehicle 3=Traffic Sign

```

Q3. Bidirectional RNN and Encoder–Decoder – Multilingual Chatbot

Problem

A multinational customer-service company needs a chatbot that translates customer queries while preserving their contextual meaning. The proposed solution is an **Encoder–Decoder sequence-to-sequence model using Bidirectional RNNs**.

Architecture

Input Sentence

↓

Tokenization & Embedding

↓

Bidirectional RNN Encoder

↓

Context Representation

↓

RNN Decoder

↓

Translated Sentence

Example

Input:

"Where is my order?"

The encoder processes the complete input sequence and generates a context representation.

The decoder then generates:

"¿Dónde está mi pedido?"

Bidirectional RNN

A normal RNN processes a sequence in one direction:

Past → Present → Future

A Bidirectional RNN processes it in two directions:

Past → Present

Future → Present

The two hidden states are combined to obtain a richer representation.

Why Bidirectional Processing Helps

Consider:

"I saw a man with a telescope."

Understanding the complete sentence requires information from both earlier and later words.

Bidirectional processing:

- Captures previous context.
- Captures future context.
- Provides richer sentence representations.
- Helps resolve contextual relationships.
- Improves translation quality.

Encoder

The encoder converts the input sentence into meaningful hidden representations.

Decoder

The decoder uses these representations to generate the translated sentence one word at a time.

Output:

```
... Epoch 1/2
2/2 ----- 10s 33ms/step - accuracy: 0.0000e+00 - loss: 2.9969
Epoch 2/2
2/2 ----- 0s 34ms/step - accuracy: 0.1500 - loss: 2.9917
Input Shape: (4, 5)
Prediction Shape: (1, 5, 20)
Bidirectional RNN Encoder-Decoder executed successfully.
Epoch 1/2
2/2 ----- 6s 35ms/step - accuracy: 0.0500 - loss: 2.9962
Epoch 2/2
2/2 ----- 0s 54ms/step - accuracy: 0.1500 - loss: 2.9904
Input Shape: (4, 5)
Prediction Shape: (1, 5, 20)
Bidirectional RNN Encoder-Decoder executed successfully.
```

Q4. BPTT and LSTM – Stock Price Prediction

Problem

A financial analytics company wants to forecast stock prices using historical market data. The system needs to learn dependencies between previous and future observations.

Proposed Architecture

Historical Stock Data

↓

Data Preprocessing

↓

Input Sequences

↓

LSTM Layer

↓

Dense Layer

↓

Predicted Stock Price

Input Data

The model can use features such as:

- Open price
- High price
- Low price
- Closing price
- Trading volume

For example:

Previous 30 days → LSTM → Next-day closing price

Why LSTM?

Traditional RNNs can have difficulty learning long-term dependencies because of the **vanishing-gradient problem**.

LSTM solves this using memory cells and gates.

The three major gates are:

1. **Forget Gate** – decides what information should be removed.
2. **Input Gate** – decides what new information should be stored.
3. **Output Gate** – decides what information should be used as output.

BPTT – Backpropagation Through Time

BPTT trains the LSTM by treating the recurrent network as a sequence of connected time steps.

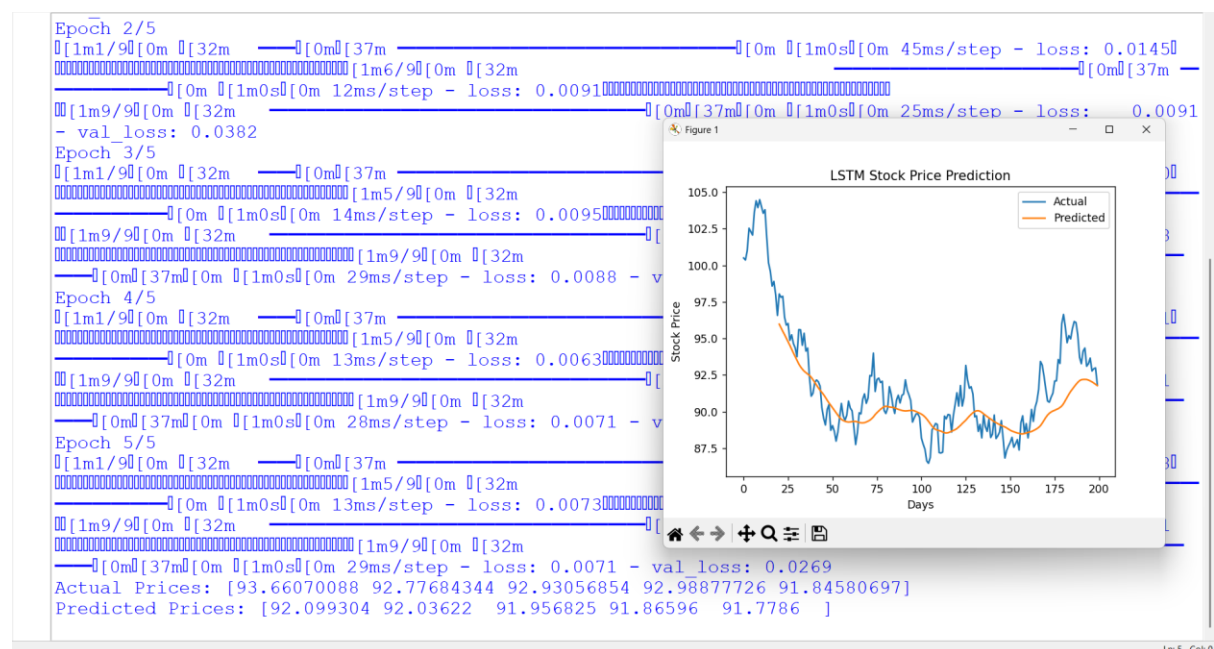
Process:

1. Input historical stock data.
2. LSTM processes each time step.
3. Generate predicted price.
4. Calculate prediction error.
5. Propagate the error backward through the time steps.
6. Calculate gradients.
7. Update the weights.
8. Repeat for multiple epochs.

Why LSTM is Better Than Traditional RNN?

Feature	Traditional RNN	LSTM
Long-term dependency	Poorer	Better
Vanishing gradient	More affected	Reduced
Memory control	Limited	Cell + gates
Financial time series	Less suitable	More suitable
Historical dependency	Limited	Strong

Output:



Q5. Integrated Deep Learning Solution – Automatic Video Captioning

The problem requires combining **Transfer Learning** for visual feature extraction with an **LSTM Encoder–Decoder** for caption generation.

Proposed Architecture

Uploaded Video



Frame Extraction



Pre-trained CNN / Transfer Learning



Visual Feature Vectors



LSTM Encoder



Context Representation



LSTM Decoder



Word-by-Word Caption Generation



Final Caption

Step 1: Video Frame Extraction

The uploaded video is divided into individual frames.

Example:

Video → Frame 1, Frame 2, Frame 3, ... Frame N

Step 2: Transfer Learning

Use a pre-trained CNN such as **ResNet50** or **VGG16**.

The CNN extracts visual information from each frame, such as:

- Objects
- People
- Background
- Actions
- Shapes
- Visual patterns

The final classification layer of the CNN can be removed so that the network works as a feature extractor.

Step 3: LSTM Encoder

The extracted visual features are given to the LSTM encoder.

The encoder learns the temporal relationship between video frames.

For example:

Frame 1 → Frame 2 → Frame 3 → Frame 4

The LSTM understands that these frames form a continuous event.

Step 4: LSTM Decoder

The decoder generates the caption sequentially.

For example:

<START> → A → man → is → playing → football → <END>

Each generated word becomes an input for predicting the next word.

Role of Each Component

Component	Role
Video	Provides visual information
Frame Extraction	Converts video into individual frames
Pre-trained CNN	Extracts visual features
Transfer Learning	Reuses knowledge from a large dataset

Component	Role
LSTM Encoder	Learns temporal information
LSTM Decoder	Generates caption
Softmax	Selects the next word

Why This Architecture Works

Transfer Learning provides powerful visual representations without requiring the CNN to be trained completely from scratch.

LSTM Encoder understands how the visual content changes across frames.

LSTM Decoder converts the learned visual-temporal representation into natural-language captions.

Example

Input video:

A person riding a bicycle in a park

Generated caption:

"A person is riding a bicycle."

Output:

```
... /tmp/ipykernel_1623/4207898567.py:9: UserWarning: `input_shape` is undefined or non-square, or `rows` is not in [96, 128, 160, 192, 224]. Weights for input shape (224, 224)
cnn=MobileNetV2(weights="imagenet",include_top=False,pooling="avg",input_shape=(64,64,3))
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
9406464/9406464 0s 0us/step
WARNING:tensorflow:6 out of the last 6 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x789ee50e3380> triggered tf.func
Epoch 1/2
2/2 3s 37ms/step - accuracy: 0.0000e+00 - loss: 3.9629
Epoch 2/2
2/2 0s 37ms/step - accuracy: 0.2000 - loss: 3.6807
Video Feature Shape: (4, 5, 1280)
Prediction Shape: (1, 5, 50)
Predicted Word IDs: [1 1 1 1 1]
Video Captioning Model Executed Successfully.
```